Complete reference for Flutter developers — FCM Push, In-App Inbox, Live Badge Count, Session Reminders

1. How It All Works (Big Picture)

Every business event triggers three simultaneous actions:

FCM Push	
Firebase push notification to device	
Even when app is closed/background	
SignalR NotificationCount	
Live badge count update via WebSocket	
Instantly while app is open	
Notification Inbox (REST)	
Stored in DB, fetchable anytime	
On demand via API	

All three happen automatically on booking created, cancelled, session started, etc. Just listen to FCM + SignalR and call the Inbox API when the user opens the notification screen.

2. FCM Token — Save at Login

Pass the Firebase device token in the login request body. Server stores it and sends push notifications to the device.

Get FCM Token in Flutter

```
final fcmToken = await FirebaseMessaging.instance.getToken();
// pass in login API call below
```

Healer Login — POST /api/v1/healer/verify-otp

```
{
  "mobileNumber": "9876543210",
  "otp": "123456",
  "timeZone": "Asia/Kolkata",
  "fcmToken": "firebase_device_token"
}
```

Patient Mobile Login — POST /api/v1/patient/verify-otp

```
{
  "mobileNumber": "9876543210",
  "otp": "123456",
  "timeZone": "Asia/Kolkata",
  "fcmToken": "firebase_device_token"
}
```

Patient Email Login — POST /api/v1/patient/verify-email-otp

```
{
  "email": "patient@email.com",
  "code": "123456",
  "timeZone": "Asia/Kolkata",
  "fcmToken": "firebase_device_token"
}
```


Add fcmToken this in this all mention api

3. Notification Inbox API

All endpoints require Authorization: Bearer <token>. Works for both Healer and Patient — returns only that user's notifications.

Get List GET **/api/v1/notifications**

Query params: page=1, pageSize=20, unreadOnly=false


```
{
  "totalCount": 42,
  "unreadCount": 5,
  "page": 1,
  "pageSize": 20,
  "items": [
    {
      "id": "uuid-here",
      "title": "Booking Confirmed",
      "body": "Your session with Dr. Riya is confirmed.",
      "screen": "booking_detail",
      "data": "{\"bookingId\":\"abc123\"}",
      "isRead": false,
      "readAt": null,
      "createdDate": "2025-06-18T10:30:00Z"
    }
  ]
}
```


Get Unread Count GET **/api/v1/notifications/unread-count**


```
{ "count": 5 }
```


Call this on app startup to set the initial badge count before SignalR connects.

Mark One Read PUT **/api/v1/notifications/{id}/read**


```
{ "message": "Marked as read.", "unreadCount": 4 }
```

Mark All Read PUT **/api/v1/notifications/read-all**

```
{ "message": "5 notification(s) marked as read.", "unreadCount": 0 }
```

Delete DELETE **/api/v1/notifications/{id}**

```
{ "message": "Notification deleted." }
```

4. Live Badge Count via SignalR

Every time a notification is created, the server pushes NotificationCount to that user via SignalR. Flutter listens and updates the badge instantly.

Event: NotificationCount — Payload

```
{ "count": 7 }
```

Flutter — Connect and Listen

```
final hub = HubConnectionBuilder()
  .withUrl("https://your-api.com/hubs/session",
    options: HttpConnectionOptions(
      accessTokenFactory: () async => token,
    ))
  .build();

// Live badge count
hub.on("NotificationCount", (args) {
  badgeCount.value = args?[0]['count'] as int? ?? 0;
});

await hub.start();
```

Full Flow: App opens → call GET /unread-count (initial badge) → connect SignalR → listen NotificationCount → badge auto-updates on every new notification.

5. All Notification Events

New Booking Created	Healer & Patient	booking_detail
Booking Cancelled by Patient	Healer	booking_detail
Booking Cancelled by Admin	Healer & Patient	booking_detail
Session Started	Patient	

session_active
Session Completed
Patient
booking_detail
Live Request Accepted
Patient
live_join
Live Request Rejected
Patient
home
Live Session Ended
Patient
booking_detail
Waitlist — Your Turn
Patient
live_session
Healer Earnings Credited
Healer
wallet_earnings
Onboarding Approved
Healer
dashboard
Onboarding Rejected
Healer
healer_onboarding

This is all screen value not screen name

booking_detail
booking_detail
booking_detail
session_active
booking_detail
live_join
home
booking_detail
live_session
wallet_earnings
dashboard
healer_onboarding

Navigate on FCM Tap

```
FirebaseMessaging.onMessageOpenedApp.listen((message) {
  final screen = message.data['screen'] ?? 'home';
  switch (screen) {
    case 'booking_detail': go(BookingDetailScreen(id: message.data['bookingId'])); break;
    case 'live_join':      go(LiveJoinScreen()); break;
```

```
    case 'wallet_earnings': go(WalletScreen()); break;
    case 'dashboard':       go(DashboardScreen()); break;
    default:                go(HomeScreen());
  }
});
```

6. Session Reminders (Automatic)

For Personal and Group sessions, the server auto-schedules SignalR reminders. Flutter just listens — nothing extra to do.

60 min before session
SessionReminder
Patient & Healer
30 min before session
SessionReminder
Patient & Healer
Note: Live sessions do NOT get scheduled reminders. Only Personal and Group sessions get T–60 and T–30 reminders.

SessionReminder Payload

```
{
  "bookingId": "abc-123",
  "scheduledTime": "2025-06-18T14:00:00Z",
  "minutesBefore": 60,
  "message": "Your session starts in 60 minutes",
  "screen": "booking_detail"
}
```

"screen": "booking_detail" when getting this then navigate to session details or booking details screen

Flutter — Listen

```
hub.on("SessionReminder", (args) {
  final data = args?[0];
  showBanner(data['message'], onTap: () => go(BookingDetailScreen(id: data['bookingId'])));
});
```

7. Notification Screen — Flutter Example

```
class NotificationScreen extends StatefulWidget {
  @override
  State<NotificationScreen> createState() => _State();
}

class _State extends State<NotificationScreen> {
  List items = [];

  @override
  void initState() {
```

```
    super.initState();
    _load();
  }

  Future _load() async {
    final res = await api.get('/notifications?page=1&pageSize=20');
    setState(() => items = res['items']);
  }

  Future _tap(Map n) async {
    await api.put('/notifications/${n["id"]}/read');
    final data = jsonDecode(n['data'] ?? '{}');
    navigateToScreen(n['screen'], data);
  }

  Future _markAllRead() async {
    await api.put('/notifications/read-all');
    badgeCount.value = 0;
    setState(() { for (var n in items) n['isRead'] = true; });
  }
}
```

8. Quick Reference

API Endpoints

GET	/api/v1/notifications	Paginated list
GET	/api/v1/notifications/unread-count	Badge count on app open
PUT	/api/v1/notifications/{id}/read	Mark one read
PUT	/api/v1/notifications/read-all	Mark all read
DELETE	/api/v1/notifications/{id}	Delete one

SignalR Events

NotificationCount	{"count": 5}	Live badge update on every new notification

SessionReminder
{"bookingId","minutesBefore","message","screen"}
T−60 & T−30 reminders

Screen Values

booking_detail session_active live_session live_join wallet_earnings dashboard healer_onboarding home

9. Flutter Developer Checklist

- Get FCM token at app startup and pass it in every login API call
- Call GET /notifications/unread-count on app open — set initial badge
- Listen to NotificationCount — update badge count in state in home screen dashboard after connectiong socket
- Listen to SessionReminder — show in-app banner show proper message
- Handle FirebaseMessaging.onMessageOpenedApp — read data["screen"] and navigate
- Handle FirebaseMessaging.onMessage (foreground) — show local notification or banner
- Build Notification List screen — call GET /notifications
- On notification tap — call PUT /notifications/{id}/read then navigate to screen
- Add "Mark All Read" button — call PUT /notifications/read-all give one text in notification screen if not exist notification screen then create screen and notification as per my app design
- Parse data field (JSON string) with jsonDecode to get bookingId etc.